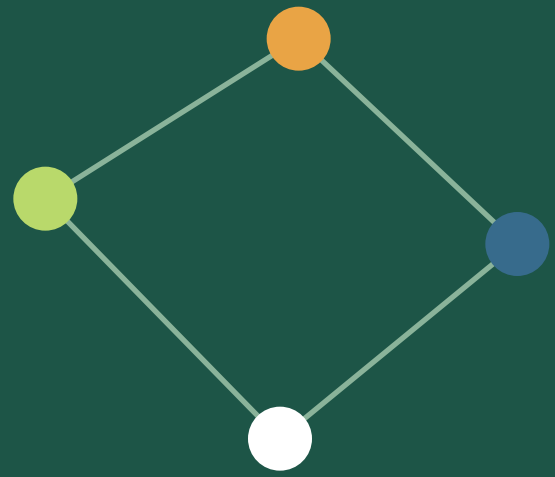
VÉLOVILLE HAUTS-DE-FRANCE / E.2

Des données fiables à la décision opérationnelle

Collecter, fiabiliser et restituer les données d'un service de vélos en libre-service.



120 STATIONS

PRÉSENTÉ PAR

Aurélien Mouille

API REST POSTGRESQL STREAMLIT

Un service régional, trois sources à fiabiliser

120
stations

1 800
vélos

~9 000
trajets / jour

Périmètre V1

Stations · trajets · incidents
Deux profils : exploitant et direction
Météo et anticipation reportées.

Enjeu

Transformer des exports hétérogènes du système de billettique en données contrôlées, consultables et utiles aux équipes terrain comme à la direction.

Les volumes de la mission sont des ordres de grandeur du commanditaire ; l'extrait fourni pour la recette compte 2 030 trajets bruts.

Un flux lisible, des responsabilités séparées

01 CSV

stations
trajets
incidents

→

02 FIABILISER

Pandas
règles pures
journaliser

→

03 STOCKER

PostgreSQL
clés & contraintes

→

04 EXPOSER

FastAPI
JWT · pagination

→

05 RESTITUER

Streamlit
tables · métriques

Contrat stable

API REST documentée, modèles SQLAlchemy, schéma relationnel et enveloppes paginées cohérentes.

Deux niveaux d'accès

Exploitant : lecture et mise à jour d'incident. Direction : lecture et indicateurs.

Fiabiliser avant de charger

SOURCE	LIGNES	IMPORTÉES	ÉCARTS TRACÉS
stations.csv	120	120	aucun
trajets.csv	2 030	1 998	20 rejetées · 12 doublons
incidents.csv	180	177	3 rejetées

60 événements de journal

35 rejets / doublons et 25 avertissements de normalisation.

Import idempotent

Deux exécutions : 120 stations, 1 998 trajets, 177 incidents. Aucune arrivée orpheline.

Protéger les actions, pas seulement les écrans

Authentification

POST /auth/login vérifie le hash bcrypt et émet un JWT HS256.
Secret et durée configurés par variables d'environnement.
Jeton transmis dans Authorization: Bearer.

Contrôle par rôle

Exploitant : lecture des ressources et transition d'incident autorisée.
Direction : lecture et indicateurs ; modification refusée en 403.

PREUVES DE RECETTE

401	anonyme / jeton invalide
403	direction sur PUT incident
422	transition non autorisée
200	santé et connexion

REC-06 : le refus de la direction ne modifie pas l'incident ; la mise à jour exploitant est ensuite annulée par rollback en base de test.

Quatre écrans, des états explicites

Connexion

Formulaire · rôle affiché

Stations

Filtre commune · pagination serveur

Incidents

Filtre statut · action exploitant

Indicateurs

3 métriques · 3 tableaux

GET /sante vérifié avant session et cache

Un chiffre métier rapproché de la base

24—30 AOÛT 2026

1 998

trajets sur la période

API total = somme SQL par jour

JOUR	TRAJETS	CONTRÔLE
24 août	**329**	**conforme**
25 août	**310**	**conforme**
26 août	**306**	**conforme**
27 août	**300**	**conforme**
28 août	**315**	**conforme**
29 août	**218**	**conforme**
30 août	**220**	**conforme**

Même jours, mêmes comptages, ordre croissant. Total de `/trajets` : 1 998.

Tester au bon niveau, garder une preuve

20	82 %	10	9 / 10
tests réussis	couverture api/services	cas de recette rejoués	cas OK après correction

Unitaires

Fiabilisation des CSV et calcul des indicateurs, avec cas nominaux et rejets.

Intégration et recette

TestClient pour sécurité, rôles et pagination ; navigateur pour les états front et la panne API.

Mesure locale documentée dans README.md · 14/10/2026 · 20 tests · couverture des services : 82 %.

Dix cas exécutés, une réserve restante

BILAN FINAL

9 OK

1 KO · 0 NT

10 cas prévus et exécutés

ANO-002 CORRIGÉE

ANO-001 OUVERTE

REC-10 · corrigé et rejoué OK

Contrôle de /sante avant session et cache : F5 hors ligne affiche le message API ; retour confirmé après redémarrage.

REC-04 · seul KO restant

Le mot de passe `mauvais` déclenche 422 (trop court), alors que l'utilisateur inconnu renvoie 401. Avec `mauvais123`, les deux réponses sont le même 401.

Réserve non bloquante : corriger la donnée du scénario REC-04 puis rejouer ce cas.

Une base solide, des limites explicites

Lancer en local

API : `python -m uvicorn api.main:app --host 127.0.0.1 --port 8000`
Front : `python -m streamlit run front/app.py`
Configuration dans `.env`.

Maintenance préventive

Sauvegarde PostgreSQL horodatée par `scripts/sauvegarde.sh`.
Restauration documentée dans la procédure de déploiement.

LIMITES ET AXES D'AMÉLIORATION

- Rendre la contrainte de capacité cohérente avec RG1 et contrôler la relation usager / référence.
- Automatiser la purge des trajets de plus de 13 mois et journaliser toutes les normalisations.
- Placer le déploiement réseau derrière HTTPS ; météo et anticipation restent hors périmètre V1.

Merci · Questions du jury